FBLDetect: Feedback Loop Detection in ODE models in Matlab

Table of Contents

Installation

Download and unzip the content of the folder 'FBLDetect_for_Matlab'. Within the Matlab session, navigate to this folder and work there or add the path of the folder to Matlab's search path for the current Matlab session by Matlab's `addpath` function. Deending on where you stored the folder, its name could be something like '/Users/Desktop/FBLDetect' on Mac or 'C:\matlab\FBLDetect' on Windows.

```
% Retrieve the FBLDetect folder location when having navigated to the
 folder
% within the Matlab session.
FBLDetect_Folder_Name = pwd;
% Add this location to Matlab's searchpath.
addpath(FBLDetect_Folder_Name)
```

Attention: The FBLDetect content will be added only for the current session. If restarting Matlab, you will have to perform this procedure again unless the FBLDetect folder is stored in a location that already belongs to Matlab's searchpath (view it with Matlab's `path` function), or you navigate to the folder and work within the folder itself.

In brief and quick start

The function suite FBLDetect enables determining all feedback loops of an ordinary differential equation (ODE) system at user-defined values of the model parameters and of the modelled variables.

The following call reports (up to 10) feedback loops for an ODE system determined by a function, here `func_POSm4`, at variable values `s_star` (here these are all equal to 1, column vector). The function values are only allowed to depend on the variable values, other parameters and time have to be fixed.

```
s_star=[1,1,1,1]';
```

```
klin=ones(1,8); knonlin=[2.5,3];
loop_list=find_loops_vset(@(x)func_POSm4(1,x,klin,knonlin),s_star,10);
disp(loop_list{1})
first_loop=loop_list{1}.loop{1}
```

loop	length	sign
{1×5 double}	4	-1
{1×4 double}	3	1
{1×2 double}	1	-1
{1×2 double}	1	-1
{1×2 double}	1	-1
{1×2 double}	1	-1

```
first_loop =

     4     1     2     3     4
```

Introduction

Ordinary differential equation (ODE) models are used frequently to mathematically represent biological systems. Feedback loops are important regulatory features of biological systems and can give rise to different dynamic behavior such as multistability for positive feedback loops or oscillations for negative feedback loops.

The feedback loops in an ODE system can be detected with the help of its Jacobian matrix, the matrix of partial derivatives of the variables. It captures all interactions between the variables and gives rise to the interaction graph of the ODE model. In this graph, each modelled variable is a node and non-zero entries in the Jacobian matrix are (weighted) edges of the graph. Interactions can be positive or negative, according to the sign of the Jacobian matrix entry.

Directed path detection in this graph is used to determine all feedback loops (in graphs also called cycles or circuits) of the system. They are marked by a set of directed interactions forming a chain in which only the first and the last node (variable) is the same. Thereby, self-loops (loops of length one) can also occur.

FBLDetect allows for detection of all loops of the graph and also reports the sign of each loop, i.e. whether it is a positive feedback loop (the number of negative interactions is even) or a negative feedback loop (the number of negative interactions is uneven). The output is a table that captures the order of the variables forming the loop, their length and the sign of each loop.

Except for solving the ODE system to generate variable values of interest which requires the optimization toolbox, only Matlab base functions are employed and no further toolboxes are required. The algorithms rely on Christopher Maes' implementation of Johnson's algorithm for path detection (find_elem_circuits.m on github) and on Brandon Kuczenski's implementation of Tarjan's algorithm for detecting strongly connected components (tarjan.m, obtained from Matlab File Exchange).

Solving the ODE model to generate variable values of interest

```
klin=[165,0.044,0.27,550,5000,78,4.4,5.1];
```

```
knonlin=[0.3,2];
[t,sol]=ode15s(@(t,x)func_POSm4(t,x,klin,knonlin),[0,50],ones(1,4));
s_star=sol(end,:); %the last point of the simulation is chosen
```

First, the ODE model is solved for a certain set of parameters, `klin`, `knonlin`, with a numerical solver in order to determine values of interest `s_star` for the modelled variables of the ODE system. These could be steady state values, values at a specific point in time (e.g. after a stimulus) or even a set of parameter values (see section Determining loops over a set of variable values). Thus, you can skip this step if you already have a point of interest in state space, or if you want to use dummy values such as `s_star=1:4`.

Calculating the Jacobian matrix

The supplied functions `numerical_jacobian()` and `numerical_jacobian_complex()` can be used to determine numerically the Jacobian matrix of an ODE system at a certain set of values for the variables, `s_star`. The approach is that of finite differences (with real step) or complex step approach, the latter of which is supposed to deliver more exact results [Martins et al., 2003] and relies on the native implementation of complex numbers in Matlab. The input function handle `f` (in the example derived from `func_POSm4`, [Baum et al., 2016]) points to a function that defines the time derivatives of the modelled variables as a vector: `f_i(s)=dS_i/dt`. When supplied to `numerical_jacobian_complex()`, it is allowed to depend only on the values of the variables; other dependencies, e.g. on parameter values, should be removed by chosing fixed values.

```
klin=[165,0.044,0.27,550,5000,78,4.4,5.1];
knonlin=[0.3,2];
j_matrix=numerical_jacobian_complex(@(s)func_POSm4(1,s,klin,knonlin),...
s_star);
signed_jacobian=sign(j_matrix)
```

```
signed_jacobian =

    -1     0     0    -1
     1    -1     0     1
     0     1    -1     0
     0     0     1    -1
```

The (i,j)th entry of the Jacobian matrix denotes the partial derivative of variable `S_i` with respect to variable `S_j`, `J_ij=delta S_i/delta S_j`, which is positive if `S_j` has a direct positive effect on `S_i`, negative if `S_j` has a direct negative effect on `S_i` and zero if `S_j` does not have a direct effect on `S_i`.

Computing all feedback loops and useful functions for loop search

The Jacobian matrix is used to compute feedback loops in the generated interaction graph. For this, the default function is find_loops.m, in that strongly connected components are determined to reduce runtime. For smaller systems, the function find_loops_noscc.m skips this step and thus can be faster. The optional second input argument sets an upper limit to the number of detected and reported loops and thus can prevent overly long runtime (but also potentially not all loops are returned).

```
loop_list=find_loops(j_matrix)
```

```
loop_list =

  6×3 table

        loop          length    sign
    ______________    ______    ____

    {1×5 double}        4         -1
    {1×4 double}        3          1
    {1×2 double}        1         -1
    {1×2 double}        1         -1
    {1×2 double}        1         -1
    {1×2 double}        1         -1
```

Single loops can be examined by entering the corresponding entry.

```
for i=1:6
    disp(loop_list.loop{i})
end
```

```
     4     1     2     3     4

     4     2     3     4

     1     1

     2     2

     3     3

     4     4
```

The function loop_summary.m provides a convenient report on total number of loops, subdivided by their lengths and signs.

```
disp(loop_summary(loop_list))
```

```
                length_1    length_2    length_3    length_4
               _________    _________    _________    _________

    total          4            0            1            1
    negative       4            0            0            1
    positive       0            0            1            0
```

One can filter the loop list for loops containing specific variables, for example the one with index 2:

```
noi = 2;
loops_with_node2=loop_list(cellfun(@(z)
 ismember(noi,z),loop_list.loop),:)


loops_with_node2 =
```

```
  3×3 table

        loop          length    sign
    ______________    ______    ____

    {1×5 double}        4         -1
    {1×4 double}        3          1
    {1×2 double}        1         -1
```

Search loop list for loops containing specific edges defined by the indices of the ingoing and outgoing nodes. This example returns the indices of all loops with a regulation of node 3 by node 2. These are only two here.

```
loop_edge_ind=find_edges(loop_list,2,3);
loops_with_edge_2_to_3=loop_list(loop_edge_ind,:)
for i=1:2
    disp(loops_with_edge_2_to_3.loop{i})
end
```

```
loops_with_edge_2_to_3 =

  2×3 table

        loop          length    sign
    ______________    ______    ____

    {1×5 double}        4         -1
    {1×4 double}        3          1

     4     1     2     3     4

     4     2     3     4
```

Saving and reading loop lists from files can be done using Matlab's `save()` and `load()` functions. They keep the correct data format, but objects have to be retrieved from a struct.

```
save('loop_list_example.mat', 'loops_with_node2')
loaded_loops_with_node2=load('loop_list_example.mat').loops_with_node2
```

```
loaded_loops_with_node2 =

  3×3 table

        loop          length    sign
    ______________    ______    ____

    {1×5 double}        4         -1
    {1×4 double}        3          1
    {1×2 double}        1         -1
```

Reading and writing loop lists to tabular format can be performed via Matlab's `writetable()` and `readtable()` functions. Here, we choose tabs as delimiters. Note that the formatting is lost.

```
writetable(loops_with_node2,'loop_list_example.txt','Delimiter','\t')
loops_with_node2_readin = readtable('loop_list_example.txt')
```

```
loops_with_node2_readin =

  3×7 table

    loop_1    loop_2    loop_3    loop_4    loop_5    length    sign
    ______    ______    ______    ______    ______    ______    ____

      4         1         2         3         4         4        -1
      4         2         3         4       NaN         3         1
      2         2       NaN       NaN       NaN         1        -1
```

Computing feedback loops over multiple sets of variable values of interest

In this example of a model of the bacterial cell cycle [Li et al., 2008], we demonstrate how feedback loops can be determined over multiple sets of variable values. Here, we focus on the solution of the ODE systems along the time axis (provided in 'li08_solution.txt').

```
%load sets of variable values (solution to the ODE over time)
sol = readmatrix('li08_solution.txt');
[loop_rep,loop_rep_index,jac_rep,jac_rep_index]=find_loops_vset(...
    @(x)func_li08(0,x), sol(:,2:end)', 1e5, false);
```

The solutions give rise to seven different loop lists.

```
disp(length(loop_rep))
```

```
     7
```

Here, two examples of resulting loop lists are given (without self-loops).

```
loop_list_2=loop_rep{2}(loop_rep{2}.length>1,:)
loop_list_7=loop_rep{7}(loop_rep{7}.length>1,:)
```

```
loop_list_2 =

  3×3 table

        loop          length    sign
    ______________    ______    ____

    {1×4 double}        3        -1
    {1×3 double}        2         1
    {1×3 double}        2        -1
```

```
loop_list_7 =

  12×3 table

       loop         length    sign
    ____________    ______    ____

    {1×3 double}      2        -1
    {1×4 double}      3         1
    {1×3 double}      2        -1
    {1×6 double}      5         1
    {1×5 double}      4        -1
    {1×4 double}      3        -1
    {1×3 double}      2         1
    {1×7 double}      6         1
    {1×4 double}      3        -1
    {1×5 double}      4        -1
    {1×9 double}      8        -1
    {1×8 double}      7         1
```

These results could be plotted along the solution, e.g. by indicating a certain background color for certain specific loop lists, and analyzed further to discover reasons of changing loops. Please note that in order to obtain the sample solution in `li08_solution.txt` also event functions are required; the solution cannot be retrieved from integrating `func_li08` alone. Please refer to the model's publication [Li et al., 2008] for details.

Comparing two loop lists

We might want to compare loops of two systems, e.g. as obtained in the example above. Thereby, loop indices in both systems should point to the same variables, otherwise a meaningful comparison is not possible. This could also be the case for example if we examine a system in which only one regulation has changed (for example the positive feedback chain model vs. the negative feedback chain model), or if regulations changed within one system when determining loops for multiple sets of variables of interest (along a dynamic trajectory, at different steady states of the system).

```
% Function with positive regulation
klin=[165,0.044,0.27,550,5000,78,4.4,5.1];
knonlin=[0.3,2];
j_matrix=numerical_jacobian_complex(@(s)func_POSm4(1,s,klin,knonlin),...
    s_star);
loop_list_pos=find_loops(j_matrix);

% Function with negative regulation. The altered regulation affects
% two entries of the Jacobian matrix. Parameter values and set of
 variable
% values remain identical.
j_matrix_neg=j_matrix;
j_matrix_neg(1:2,4)=-j_matrix(1:2,4);
loop_list_neg=find_loops(j_matrix_neg);

% compute comparison
```

```
[ind_a_id,ind_a_switch,ind_a_notin,ind_b_id,ind_b_switch]=...
    compare_loop_list(loop_list_pos,loop_list_neg);
```

Only the four self-loops remain identical in both systems.

```
disp(loop_list_pos(ind_a_id,:))
for i=1:4
    disp(loop_list_pos.loop{ind_a_id(i)})
end
```

```
        loop         length    sign
    _____________    _______    ____

    {1×2 double}       1         -1
    {1×2 double}       1         -1
    {1×2 double}       1         -1
    {1×2 double}       1         -1

     1     1

     2     2

     3     3

     4     4
```

These are the corresponding loops in the negatively regulated system.

```
disp(loop_list_neg(ind_b_id,:))
for i=1:4
    disp(loop_list_neg.loop{ind_b_id(i)})
end
```

```
        loop         length    sign
    _____________    _______    ____

    {1×2 double}       1         -1
    {1×2 double}       1         -1
    {1×2 double}       1         -1
    {1×2 double}       1         -1

     1     1

     2     2

     3     3

     4     4
```

Two loops are the same in both systems but they have switched their signs.

```
loops_switch_pos=loop_list_pos(ind_a_switch,:)
for i=1:2
```

```
    disp(loops_switch_pos.loop{i})
end
loops_switch_neg=loop_list_neg(ind_b_switch,:)
for i=1:2
    disp(loops_switch_neg.loop{i})
end
```

```
loops_switch_pos =

  2×3 table

        loop          length    sign
    ______________    ______    ____

    {1×5 double}        4         -1
    {1×4 double}        3          1

     4     1     2     3     4

     4     2     3     4


loops_switch_neg =

  2×3 table

        loop          length    sign
    ______________    ______    ____

    {1×5 double}        4          1
    {1×4 double}        3         -1

     4     1     2     3     4

     4     2     3     4
```

All loops in the positively regulated system do also occur in the negatively regulated system, i.e. ind_a_notin is empty.

```
ind_a_notin
```

```
ind_a_notin =

  0×1 empty double column vector
```

List of functions in FBLDetect - Alphabetical overview

- `compare_loop_list` - compare two loop lists

- `find_edges` - find loops with a certain edge in a loop list
- `find_loops_noscc` - detect feedback loops from a Jacobian or adjacency matrix without determining strongly connected components, suitable for smaller or densely connected systems
- `find_loops_vset` - detect feedback loops from a function and sets of values for the modelled variables
- `find_loops` - detect feedback loops from a Jacobian or adjacency matrix
- `loop_summary` - summarize loops list contents by lengths and signs
- `numerical_jacobian_complex` - finite difference numerical determination of the Jacobian matrix with a complex step, provides higher accuracy than `numerical_jacobian`
- `numerical_jacobian` - finite difference numerical determination of the Jacobian matrix
- `sort_loop_index` - sort the reported loops to always start with the lowest node index

References

Baum K, Politi AZ, Kofahl B, Steuer R, Wolf J. Feedback, Mass Conservation and Reaction Kinetics Impact the Robustness of Cellular Oscillations. PLoS Comput Biol. 2016;12(12):e1005298.

Brandon Kuczenski (2018). tarjan(e) (https://www.mathworks.com/matlabcentral/fileexchange/50707-tarjan-e), MATLAB Central File Exchange. Retrieved August 14, 2018.

Li S, Brazhnik P, Sobral B, Tyson JJ. A Quantitative Study of the Division Cycle of Caulobacter crescentus Stalked Cells. Plos Comput Biol. 2008;4(1):e9.

Christopher Maes (2011). find_elem_circuits.m https://gist.github.com/cmaes/1260153

Martins JRRA, Sturdza P, Alonso JJ. The complex-step derivative approximation. ACM Trans Math Softw. 2003;29(3):245–62.

Published with MATLAB® R2019b